

Tests & Quizzes

Quiz 5

[Return to Assessment List](#)

Part 1 of 1 5.0 / 5.0 Points

Question 1 of 5

1.0

1.0 Points

Miniscript is described in the slides as analogous to TypeScript in relation to JavaScript. A spending policy states: Alice can spend alone, OR Bob can spend after 30 days, OR Alice and Bob together can spend anytime. When this policy is compiled to Bitcoin Script, which opcodes from the slides are necessarily involved to enforce the time constraint on Bob's spending path?

- ✓ A. OP_CHECKLOCKTIMEVERIFY and OP_IF
- ✓ B. OP_CHECKSEQUENCEVERIFY with OP_CHECKMULTISIG only
- ✓ C. OP_CHECKLOCKTIMEVERIFY with OP_CHECKSIG only
- ✓ D. OP_CHECKSEQUENCEVERIFY and OP_IF with OP_ELSE and OP_ENDIF

Answer Key: D

Question 2 of 5

1.0

1.0 Points

Transaction malleability in legacy Bitcoin was possible because the txid was computed by hashing the fully serialized transaction including the scriptSig. Which specific property of ECDSA signatures made this exploitable, and what did SegWit do structurally to fix it?

- ✓ A. ECDSA signatures have multiple valid DER encodings for the same (r, s) pair such as adding or removing leading zeros; SegWit excluded the witness data from the txid computation by moving signatures to a separate witness field
- ✓ B. ECDSA nonce reuse allowed signature forgery; SegWit replaced ECDSA with Schnorr signatures
- ✓ C. ECDSA public keys could be compressed or uncompressed interchangeably; SegWit mandated compressed keys only
- ✓ D. ECDSA signatures could be replayed across transactions; SegWit added a sequence number to the witness field

Answer Key: A

Question 3 of 5

1.0

1.0 Points

In a P2WPKH transaction, the scriptPubKey stored in the output is 00 14 <20-byte-hash>. When this output is being spent, Bitcoin internally constructs a virtual script to execute. What exactly is this virtual script and where does the 20-byte value come from?

- ✓ A. OP_SHA256 <20-byte-pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG, where the 20-byte value is expanded from the witness field and used for verification
- ✓ B. OP_DUP OP_HASH160 <20-byte-pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG, where the 20-byte value is taken directly from the scriptPubKey of the previous transaction output
- ✓ C. OP_HASH256 <20-byte-hash> OP_EQUALVERIFY OP_CHECKSIG, where the 20-byte value is derived by applying SHA256 twice on the public key provided in the witness
- ✓ D. OP_CHECKSIG OP_HASH160 <20-byte-hash> OP_EQUALVERIFY, where the 20-byte value is reconstructed from the witness data and validated against the transaction input

Answer Key: B

Question 4 of 5

1.0

1.0 Points

A developer is auditing a SegWit transaction and observes an input with an empty scriptSig. The corresponding scriptPubKey of the output being spent starts with the SegWit version byte 00. According to the slides, how does the node distinguish between a P2WPKH and a P2WSH spend, and what is the immediate internal consequence of this distinction?

- ✓ A. It checks the number of items in the witness stack; if there are two items, it expands a virtual P2PKH-style script; if there are more items, it performs hash validation on the final witness element
- ✓ B. It checks the Flag field in the transaction header; if the value indicates SegWit usage, it expands a virtual script for validation; otherwise, it processes the input using legacy script execution rules
- ✓ C. It checks the Marker and Flag fields together; if SegWit is detected, it executes the scriptSig and witness simultaneously; otherwise, it treats the hash as part of the Merkle tree validation process
- ✓ D. It checks the byte following the version 00 in the scriptPubKey; if it is 14 (20-byte hash), it expands a virtual P2PKH script; if it is 20 (32-byte hash), it validates the witnessScript against its SHA256 hash

Answer Key: D

Question 5 of 5

1.0

1.0 Points

In a P2WSH multisig execution, the witness field contains OP_0, followed by M signatures, followed by the witnessScript as the last item. During OP_CHECKMULTISIG execution, after popping N public keys and M signatures from the stack, there is still one item remaining at the bottom. What is this item, why is it there, and what happens to it?

- ✓ A. It is the witnessScript hash; it remains because SHA256 verification left it; it is compared against the scriptPubKey
- ✓ B. It is the integer 0 pushed by OP_0 at the start of the witness; it exists due to a known bug in OP_CHECKMULTISIG which pops one extra stack item
- ✓ C. It is the number of required signatures M; it exists because OP_2 pushed it; it is verified against the actual count of valid signatures
- ✓ D. It is the Merkle root of the signatures; it exists for audit purposes; it is dropped by OP_DROP after CHECKMULTISIG

Answer Key: B